CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

REGULATION-2024

RECORD

AD4402 – BIG DATA ANALYTICS

IV SEMESTER

NAME OF STUDENT :

REGISTER NO :

YEAR / SEM / SECTION :

BATCH :

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1: To creates next generation leaders by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.

IM2: To transform lives through deployment of emerging technology, novelty and sustainability.

IM3: To inculcate human values and ethical principles to cater the societal needs.

IM4: To contributes towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

Department of Artificial Intelligence and Data Science

Vision of the Department:

To Achieve excellent standards of quality-education by using the latest tools, nurturing collaborative culture and disseminating customer oriented innovations to relevant areas of academia and industry towards serving the greater cause of society.

Mission of the Department

DM 1: To develop professionals who are skilled in the area of Artificial Intelligence and Data Science.

DM 2: To impart quality and value based education and contribute towards the innovation of computing, expert system, Data Science to raise satisfaction level of all stakeholders.

DM 3: Our effort is to apply new advancements in high performance computing hardware and software.

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

Register No:

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of work done by Mr./Ms. ……of …… semester B.Tech ……

Engineering in the…… Laboratory during the academic year …….

Staff-In-Charge **Head of the Department**

Submitted for the University Practical Examination held on…….

Internal Examiner **External Examiner**

Date: …… **Date: …….....**

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

INDEX

S. No	Date	Title	Page No.	CO's Mapped	PO's Mapped	Signature
1		HDFS Commands Map Reduce Program to show the need of Combiner		CO1	PO1, PO2, PO4, PO5	
2		Map Reduce I/O Formats. Text, Key-Value Map Reduce I/O Format –Nline, Multiline		CO2	PO1, PO2, PO4, PO5	
3		Sequence file Input / Output Formats Secondary sorting		CO2	PO1, PO2, PO3, PO5	
4A		Distributed cache & Map side Join, Reduce side Join Building		CO2	PO1, PO2, PO3, PO5	
4B		Running a spark Application word count in Hadoop		CO3	PO1, PO2, PO4, PO5	
4C		Spark Manipulating RDD		CO3	PO1, PO2, PO3, PO4, PO5	
5A		Inverted Indexing in spark (spark sequence algorithm problem)		CO4	PO1, PO2, PO3, PO5	
5B		Inverted Indexing in spark (spark implementation of matrix algorithm)		CO4	PO1, PO2, PO3, PO5	
5C		Inverted Indexing in spark (spark sparlsql programming)		CO4	PO1, PO2, PO3, PO4, PO5	
5D		Inverted Indexing in spark (Building spark streaming application)		CO4, CO5	PO1, PO2, PO3, PO4, PO5, PO12	

Exp No: 1 Date:	**HDFS Commands Map Reduce Program to show the need of Combiner**

AIM:

To practice HDFS commands and implement a MapReduce program showing the need of a combiner

Procedure:

1. Open Hadoop and ensure HDFS services are running.
2. Use hdfs dfs -ls to list files and directories.
3. Create directories using hdfs dfs -mkdir.
4. Transfer files from the local file system to HDFS using -put or -copyFromLocal.
5. View file contents with hdfs dfs -cat and check file sizes using hdfs dfs -du.
6. Move and copy files within HDFS using mv and cp.
7. Write a MapReduce program to compute maximum units with and without a combiner.
8. Compile and execute the program on sample input to observe the difference in execution

<u>HDFS Command:</u>

1. **ls:** This command is used to list all the files. Use *lsr* for recursive approach. It is useful when we want a hierarchy of a folder.
   Syntax:
   bin/hdfs dfs -ls <path>

   Example:
   bin/hdfs dfs -ls /

   It will print all the directories present in HDFS. bin directory contains executables so, *bin/hdfs* means we want the executables of hdfs particularly *dfs* (Distributed File System) commands.

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /
19/01/31 10:35:11 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 4 items
-rw-r--r--   1 suraj supergroup   13965969 2019-01-31 00:13 /input
drwxr-xr-x   - suraj supergroup          0 2019-01-31 01:30 /output
drwx------   - suraj supergroup          0 2019-01-31 00:15 /tmp
drwxr-xr-x   - suraj supergroup          0 2019-01-30 23:44 /user
```

2. **mkdir**: To create a directory. In Hadoop *dfs* there is no home directory by default. So let's first create it.
   Syntax:
   bin/hdfs dfs -mkdir <folder name>
   creating home directory:

   hdfs/bin -mkdir /user

   hdfs/bin -mkdir /user/username -> write the username of your computer

   Example:
   bin/hdfs dfs -mkdir /geeks => '/' means absolute path
   bin/hdfs dfs -mkdir geeks2 => Relative path -> the folder will be created relative to the home directory.

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -mkdir /geeks
19/01/31 10:53:43 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /
19/01/31 10:53:56 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 5 items
drwxr-xr-x   - suraj supergroup          0 2019-01-31 10:53 /geeks
-rw-r--r--   1 suraj supergroup   13965969 2019-01-31 00:13 /input
drwxr-xr-x   - suraj supergroup          0 2019-01-31 01:30 /output
drwx------   - suraj supergroup          0 2019-01-31 00:15 /tmp
drwxr-xr-x   - suraj supergroup          0 2019-01-30 23:44 /user
```

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -mkdir geeks2
19/01/31 10:59:33 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -lsr /user
lsr: DEPRECATED: Please use 'ls -R' instead.
19/01/31 10:59:51 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
drwxr-xr-x   - suraj supergroup          0 2019-01-31 10:59 /user/suraj
-rw-r--r--   1 suraj supergroup      28093 2019-01-31 00:10 /user/suraj/AFINN-111.txt
drwxr-xr-x   - suraj supergroup          0 2019-01-31 10:59 /user/suraj/geeks2
drwxr-xr-x   - suraj supergroup          0 2019-01-30 23:47 /user/suraj/insideUserChecking
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

3. **touchz**: It creates an empty file.
   Syntax:
   bin/hdfs dfs -touchz <file_path
   Example:
   bin/hdfs dfs -touchz /geeks/myfile.txt

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -touchz /geeks/myfile.txt
19/01/31 11:10:31 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -lsr /geeks
lsr: DEPRECATED: Please use 'ls -R' instead.
19/01/31 11:10:48 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

4. **copyFromLocal (or) put:** To copy files/folders from local file system to hdfs store. This is the most important command. Local filesystem means the files present on the OS.

   Syntax:

   bin/hdfs dfs -copyFromLocal <local file path> <dest(present on hdfs)>

Example: Let's suppose we have a file *AI.txt* on Desktop which we want to copy to folder *geeks* present on hdfs.

```
bin/hdfs dfs -copyFromLocal ../Desktop/AI.txt /geeks
(OR)
bin/hdfs dfs -put ../Desktop/AI.txt /geeks
```

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -put ../Desktop/AI.txt /geeks
19/01/31 11:31:02 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -lsr /geeks
lsr: DEPRECATED: Please use 'ls -R' instead.
19/01/31 11:31:21 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
-rw-r--r--   1 suraj supergroup        205 2019-01-31 11:31 /geeks/AI.txt
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

5. **cat:** To print file contents.

   Syntax:

   bin/hdfs dfs -cat <path>

   Example:

```
// print the content of AI.txt present
// inside geeks folder.
bin/hdfs dfs -cat /geeks/AI.txt ->
```

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -cat /geeks/AI.txt
19/01/31 11:33:25 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
In computer science, artificial intelligence, sometimes called machine intelligence, is intelligence demonstrated by machines, in contrast to t
he natural intelligence displayed by humans and other animals
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

6. **copyToLocal (or) get:** To copy files/folders from hdfs store to local file system.

   Syntax:

   bin/hdfs dfs -copyToLocal <<srcfile(on hdfs)> <local file dest>

   Example:

   bin/hdfs dfs -copyToLocal /geeks ../Desktop/hero

   (OR)

   bin/hdfs dfs -get /geeks/myfile.txt ../Desktop/hero

   myfile.txt from *geeks* folder will be copied to folder *hero* present on *Desktop*.

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -get /geeks/myfile.txt  ../Desktop/hero
19/01/31 11:43:34 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ ls ../Desktop/hero
myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

Note: Observe that we don't write *bin/hdfs* while checking the things present on local filesystem.

7. **moveFromLocal:** This command will move file from local to hdfs.

   Syntax:

   bin/hdfs dfs -moveFromLocal <local src> <dest(on hdfs)>

   Example:

   bin/hdfs dfs -moveFromLocal ../Desktop/cutAndPaste.txt /geeks

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -moveFromLocal ../Desktop/cutAndPaste.txt /geeks
19/01/31 12:38:38 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks
19/01/31 12:38:56 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 3 items
-rw-r--r--   1 suraj supergroup        205 2019-01-31 11:31 /geeks/AI.txt
-rw-r--r--   1 suraj supergroup         96 2019-01-31 12:38 /geeks/cutAndPaste.txt
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ ls ../Desktop
AFINN-111.txt  Deploy_hadoop_on_a_single_node_cluster_v02 (1).pdf  hero                 sentiment.jar
AI.txt         FlumeData.1440939532959                             json-simple-1.1.1.jar  worldBank
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

8. **cp:** This command is used to copy files within hdfs. Lets copy folder *geeks* to *geeks_copied*.

   Syntax:

   bin/hdfs dfs -cp <src(on hdfs)> <dest(on hdfs)>

   Example:

   bin/hdfs dfs -cp /geeks /geeks_copied

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -mkdir /geeks_copied
19/01/31 12:46:03 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -cp /geeks  /geeks_copied
19/01/31 12:46:26 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks_copied
19/01/31 12:47:10 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 1 items
drwxr-xr-x   - suraj supergroup          0 2019-01-31 12:46 /geeks_copied/geeks
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

9. **mv:** This command is used to move files within hdfs. Lets cut-paste a file *myfile.txt* from *geeks* folder to *geeks_copied*.

   Syntax:

   bin/hdfs dfs -mv <src(on hdfs)> <src(on hdfs)>

   Example:

   bin/hdfs dfs -mv /geeks/myfile.txt /geeks_copied

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks
19/01/31 12:48:42 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 3 items
-rw-r--r--   1 suraj supergroup        205 2019-01-31 11:31 /geeks/AI.txt
-rw-r--r--   1 suraj supergroup         96 2019-01-31 12:38 /geeks/cutAndPaste.txt
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -mv /geeks/myfile.txt  /geeks_copied
19/01/31 12:49:27 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks
19/01/31 12:49:42 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 2 items
-rw-r--r--   1 suraj supergroup        205 2019-01-31 11:31 /geeks/AI.txt
-rw-r--r--   1 suraj supergroup         96 2019-01-31 12:38 /geeks/cutAndPaste.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks_copied
19/01/31 12:50:00 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 2 items
drwxr-xr-x   - suraj supergroup          0 2019-01-31 12:46 /geeks_copied/geeks
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks_copied/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

10. **rmr:** This command deletes a file from HDFS *recursively*. It is very useful command when you want to delete a *non-empty directory*.

    Syntax:

    bin/hdfs dfs -rmr <filename/directoryName>

    Example:

    bin/hdfs dfs -rmr /geeks_copied -> It will delete all the content inside the directory then the directory itself.

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks_copied
19/01/31 12:58:16 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 2 items
drwxr-xr-x   - suraj supergroup          0 2019-01-31 12:46 /geeks_copied/geeks
-rw-r--r--   1 suraj supergroup          0 2019-01-31 11:10 /geeks_copied/myfile.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -rmr /geeks_copied
rmr: DEPRECATED: Please use 'rm -r' instead.
19/01/31 12:58:40 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
19/01/31 12:58:42 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes, Emptier interval = 0 minutes.
Deleted /geeks_copied
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks_copied
19/01/31 12:59:35 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
ls: `/geeks_copied': No such file or directory
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

11. **du:** It will give the size of each file in directory.
    Syntax:
    bin/hdfs dfs -du <dirName>
    Example:
    bin/hdfs dfs -du /geeks

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -du /geeks
19/01/31 13:01:57 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
205  205  /geeks/AI.txt
96   96   /geeks/cutAndPaste.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

12. **dus:**: This command will give the total size of directory/file.
    Syntax:
    bin/hdfs dfs -dus <dirName>
    Example:
    bin/hdfs dfs -dus /geeks

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -dus /geeks
dus: DEPRECATED: Please use 'du -s' instead.
19/01/31 13:02:34 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
301  301  /geeks
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

13. **stat:** It will give the last modified time of directory or path. In short it will give stats of the directory or file.
    Syntax:
    bin/hdfs dfs -stat <hdfs file>
    Example:
    bin/hdfs dfs -stat /geeks

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -stat /geeks
19/01/31 13:03:39 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
2019-01-31 07:19:29
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$
```

14. **setrep:** This command is used to change the replication factor of a file/directory in HDFS. By default it is 3 for anything which is stored in HDFS (as set in hdfs *core-site.xml*).
    Example 1: To change the replication factor to 6 for *geeks.txt* stored in HDFS.
    bin/hdfs dfs -setrep -R -w 6 geeks.txt
    Example 2: To change the replication factor to 4 for a directory *geeksInput* stored in HDFS.
    bin/hdfs dfs -setrep -R 4 /geeks
    Note: The **-w** means wait till the replication is completed. And **-R** means recursively, we use it for directories as they may also contain many files and folders inside them.

```
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -setrep -R  4 /geeks
19/01/31 13:05:31 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Replication 4 set: /geeks/AI.txt
Replication 4 set: /geeks/cutAndPaste.txt
suraj@suraj:~/hadoop-2.5.0-cdh5.3.2$ bin/hdfs dfs -ls /geeks
19/01/31 13:05:50 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applic
able
Found 2 items
-rw-r--r--   4 suraj supergroup        205 2019-01-31 11:31 /geeks/AI.txt
-rw-r--r--   4 suraj supergroup         96 2019-01-31 12:38 /geeks/cutAndPaste.txt
```

Program:

1.1 Map Reduce Program:

```
package hadoop;

import java.util.*;

importjava.io.IOException;
import java.io.IOException;

importorg.apache.hadoop.fs.Path; import
org.apache.hadoop.conf.*; import
org.apache.hadoop.io.*;
importorg.apache.hadoop.mapred.*;
import org.apache.hadoop.util.*;

public class ProcessUnits {
	//Mapper class
	public static class E_EMapper extends MapReduceBase implements
	Mapper<LongWritable ,/*Input key Type */
	Text,                                   /*Input
	value Type*/
	Text,                                   /*Output
	key Type*/ IntWritable>             /*Output
	value Type*/
	{
		//Map function
		public void map(LongWritable key, Text value, OutputCollector<Text,
		IntWritable> output,

		Reporter reporter) throws IOException {
			String line = value.toString();
			String lasttoken = null;
			StringTokenizer s = new StringTokenizer(line,"\t");
			String year = s.nextToken();

			while(s.hasMoreTokens()) {
				lasttoken = s.nextToken();
			}
			int avgprice = Integer.parseInt(lasttoken);
			output.collect(new Text(year), new
```

```
IntWritable(avgprice));
            }
        }
        //Reducer class
        public static class E_EReduce extends MapReduceBase
implements Reducer< Text, IntWritable, Text, IntWritable
> {
    //Reduce function
            public void reduce( Text key, Iterator
<IntWritable> values,
            OutputCollector<Text, IntWritable> output, Reporter reporter)
throws IOException {
                int maxavg = 30;
                int val = Integer.MIN_VALUE;

            while (values.hasNext()) {
                    if((val = values.next().get())>maxavg) {
                        output.collect(key, new IntWritable(val));
                    }
                }
            }
        }

        //Main function
        public static void main(String args[])throws Exception { JobConf
            conf = new JobConf(ProcessUnits.class);

            conf.setJobName("max_eletricityunits"); conf.setOutputKeyClass(Text.class);
            conf.setOutputValueClass(IntWritable.class);
            conf.setMapperClass(E_EMapper.class);
            conf.setCombinerClass(E_EReduce.class);
            conf.setReducerClass(E_EReduce.class);
            conf.setInputFormat(TextInputFormat.class);
            conf.setOutputFormat(TextOutputFormat.class);
            FileInputFormat.setInputPaths(conf, new Path(args[0]));
            FileOutputFormat.setOutputPath(conf, new Path(args[1]));
            JobClient.runJob(conf);
        }
    }
```

INPUT

```
2018   unit1   20
2018   unit2   40
2019   unit1   35
2019   unit2   25
2020   unit1   50
```

OUTPUT

```
2018   40
2019   35
2020   50
```

Exp No: 2 Date:	**Map Reduce I/O Formats. Text, Key-Value Map Reduce I/O Format –Nline, Multiline**

Aim

To study and implement different MapReduce input/output formats:
Text, KeyValue, NLine, MultiLine.

Procedure

1. Start Hadoop and ensure input files are placed in HDFS.
2. Implement a MapReduce job with TextInputFormat for line-by-line processing.
3. Implement KeyValueTextInputFormat by specifying a key-value delimiter.
4. Configure NLineInputFormat to assign N lines per mapper task.
5. Test MultiLineInputFormat for grouped line records such as XML or JSON.
6. Compile the Java program with Hadoop libraries.
7. Run the program with sample input data in each format.
8. Observe and compare how records are split and processed differently.

Program:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Text_input_output_example {

  public static class MyMapper extends Mapper<LongWritable, Text,

    Text, Text> { protected void map(LongWritable key, Text value,

    Context context) throws
IOException, InterruptedException {
      // Put the map logic here.
      context.write(new Text("OutputKey"), value);
    }
  }

  public static void main(String[] args) throws Exception {
    // Construct a Hadoop
    configuration.
    Configuration conf = new
    Configuration();

    Job job = Job.getInstance(conf, "Text_input_output_example");
```

```
        job.setJarByClass(Text_input_output_example.class);

        // TextInputFormat should be used as the input format class.
        job.setInputFormatClass(TextInputFormat.class);

    job.setMapperClass(MyMapper.class);

    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);

    // TextOutputFormat should be used as the output format class.
    job.setOutputFormatClass(TextOutputFormat.class);

    job.setOutputKeyClass(Text.class);
    ob.setOutputValueClass(Text.class);

    TextInputFormat.addInputPath(job, new Path("input_dir"));
    TextOutputFormat.setOutputPath(job, new Path("output_dir"));

    // Wait for the job to finish before exiting.
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

INPUT

apple banana
grapes mango
hello world

OUTPUT

OutputKey apple banana
OutputKey grapes mango
OutputKey hello world

Exp No: 3 Date:	**Map Reduce I/O Formats. Text, Key-Value Map Reduce I/O Format –Nline, Multiline**

Aim

To understand sequence file format and implement secondary sorting using composite keys.

Procedure

1. Create sequence files to store key-value data in binary format.
2. Write a custom Writable class with composite keys like (year, month).
3. Configure a partitioner to ensure same keys go to the same reducer.
4. Implement a grouping comparator to group keys by year.
5. Compile the program and run with input containing date and temperature.
6. Observe sorting behavior across reducers.
7. Compare output order with original unsorted input.
8. Verify that secondary sorting arranges records by year first and then by month/day

Program:

```
public class SecondarySortDriver extends Configured implements Tool {
public int run(String[] args) throws Exception {
        Configuration conf = getConf(); Job
        job = new Job(conf);
        job.setJarByClass(SecondarySortDriver.class);
        job.setJobName("SecondarySortDriver");

        Path inputPath = new Path(args[0]); Path
        outputPath = new Path(args[1]);
        FileInputFormat.setInputPaths(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputPath);

        job.setOutputKeyClass(TemperaturePair.class);
        job.setOutputValueClass(NullWritable.class);
```

```
            job.setMapperClass(SecondarySortingTemperatureMapper.class);
            job.setReducerClass(SecondarySortingTemperatureReducer.class);
            job.setPartitionerClass(TemperaturePartitioner.class);
            job.setGroupingComparatorClass(YearMonthGroupingComparator.class);

            boolean status = job.waitForCompletion(true);
            theLogger.info("run(): status="+status); return status
            ? 0 : 1;
        }

    /**
     * The main driver for the secondary sort MapReduce program.
     * Invoke this method to submit the MapReduce job.
     * @throws Exception when there are communication
     * problems with the job tracker.
     */
      public static void main(String[] args) throws Exception {
            // Make sure there are exactly 2 parameters if
            (args.length != 2) {
                  throw new IllegalArgumentException("Usage: SecondarySortDriver" + "
                        <input-path> <output-path>");
            }

           //String inputPath = args[0];
           //String outputPath = args[1];
           int returnStatus = ToolRunner.run(new SecondarySortDriver(), args);
            System.exit(returnStatus);
      }
}
```

Sample Run of
Hadoop
Implementation Input

```
# cat sample_input.txt
```

INPUT

2000,12,04,10
2000,11,01,20
2000,12,02,-20
2000,11,07,30
2000,11,24,-40
2012,12,21,30
2012,12,22,-20
2012,12,23,60
2012,12,24,70
2012,12,25,10
2013,01,22,80
2013,01,23,90
2013,01,24,70
2013,01,20,-10

OUTPUT

2000,11,07,30
2000,11,01,20
2000,11,24,-40

2000,12,04,10
2000,12,02,-20

2012,12,24,70
2012,12,23,60
2012,12,21,30
2012,12,25,10
2012,12,22,-20

2013,01,23,90
2013,01,22,80
2013,01,24,70
2013,01,20,-10

Exp No: 4A Date:	**Distributed cache & Map side Join, Reduce side Join Building**

Aim

To implement and compare two core data joining strategies in Hadoop MapReduce: the memory- intensive Map-side Join and the network-intensive Reduce-side Join.

Procedure

Map-side Join (In-Memory Join)
This efficient method is ideal for joining a large dataset with a small one that can fit into memory.
The process works as follows:

1. Distribute & Load: The small dataset is sent to all nodes using the Distributed Cache. Each Mapper then loads this file into an in-memory HashMap during its setup() phase for fast lookups.

2. Join in Mapper: The Mapper processes the large dataset line by line. For each record, it uses the join key to find the corresponding data in the HashMap.

3. Direct Output: The joined data is written directly from the Mapper. This approach is very fast because it bypasses the entire shuffle, sort, and reduce phase.

Reduce-side Join (Shuffle & Sort Join)
This is the standard, flexible method for joining two datasets of any size, particularly when both are large.
The process involves three main stages:

1. **Tag in Mapper**: Mappers for each dataset emit the **join key**. The value is the record's data, prefixed with a unique **tag** (e.g., 'A' or 'B') to identify which dataset it came from.

2. **Group by Shuffle**: Hadoop's **Shuffle and Sort** phase automatically groups all tagged values by their common join key and sends them to a single Reducer.

3. **Join in Reducer**: The Reducer receives the key and a list of all its associated values. It uses the tags to separate the data from each source, performs the join, and writes the final combined output.

Program:
Implementation 1: Map-side Join Code
MapSideJoinDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import java.net.URI;

public class MapSideJoinDriver {

    public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Map-Side Join");

        // Add the small file to the Distributed Cache for mappers to access

        job.addCacheFile(new URI("/input/department.txt#dept")); // Using a symlink
        'dept'

        job.setJarByClass(MapSideJoinDriver.class);
        job.setMapperClass(MapSideJoinMapper.class);

        // No Reducer is needed for this type of join
        job.setNumReduceTasks(0);
        job.setOutputKeyClass(Text.class);

        job.setOutputValueClass(Text.class);
        // Input path should be the large file (e.g., Employee.txt)

        FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }

}
```

MapSideJoinMapper.java

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
 import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
 import java.util.HashMap;

public class MapSideJoinMapper extends Mapper<LongWritable, Text, Text, Text> {

    private HashMap<String, String> deptMap = new HashMap<>();

    // setup() is called once per Mapper task before the map() method

    @Override

    protected void setup(Context context) throws IOException, InterruptedException {

        // Retrieve the cached file from the local disk of the node

        Path[] cacheFiles = context.getLocalCacheFiles();

        if (cacheFiles != null && cacheFiles.length > 0) {

        String line;

            // The symlink 'dept' created in the driver is used here

            BufferedReader reader = new BufferedReader(new FileReader("dept"));

            try {

                // Read the department file and populate the HashMap

                while ((line = reader.readLine()) != null) {

                    String[] tokens = line.split(",");

                    if (tokens.length == 2) {

                        deptMap.put(tokens[0], tokens[1]);

                    }

                }
    } finally {
```

```
            reader.close();

            }
        }
    }

            // map() is called for each line of the large file (Employee.txt)

            @Override

            protected void map(LongWritable key, Text value, Contextcontext)throws IOException,
            InterruptedException {
                String[] empTokens = value.toString().split(",");

                if (empTokens.length == 3) {

                    String empId = empTokens[0];

                    String empName = empTokens[1];

                    String deptId = empTokens[2];

                    // Lookup the department name from the HashMap

                    String deptName = deptMap.get(deptId);

                    if (deptName != null) {

                        // Output: EmployeeID -> EmployeeName, DepartmentName

                        context.write(new Text(empId), new Text(empName + "," + deptName));

                    }

                }

            }

        }
```

Implementation 2: Reduce-side Join (Shuffle & Sort Join)

ReduceSideJoinDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.MultipleInputs;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class ReduceSideJoinDriver {
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "Reduce-Side Join");
job.setJarByClass(ReduceSideJoinDriver.class);
// Use MultipleInputs to assign different Mappers to different input paths
MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class,
EmployeeMapper.class);
MultipleInputs.addInputPath(job, new Path(args[1]), TextInputFormat.class,
DepartmentMapper.class);
job.setReducerClass(JoinReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
FileOutputFormat.setOutputPath(job, new Path(args[2]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
} }
```

EmployeeMapper.java

```
import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

public class EmployeeMapper extends Mapper<LongWritable, Text, Text, Text> {

@Override

protected void map(LongWritable key, Text value, Context context) throws IOException,

InterruptedException {

  String[] tokens = value.toString().split(",");

if (tokens.length == 3) {

 String deptId = tokens[2]; // Join key

   String empData = "A," + tokens[0] + "," + tokens[1]; // Tag 'A' for employee

context.write(new Text(deptId), new Text(empData));

        }

}

}
```

DepartmentMapper.java

```
import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

public class Departmen tMapper extends Mapper<LongWritable, Text, Text, Text> {

   @Override

   protected void map(LongWritable key, Text value, Context context) throws
IOException,
```

```
        InterruptedException {
            String[] tokens = value.toString().split(",");
            if (tokens.length == 2) {
                String deptId = tokens[0];
                // Join key
                String deptName = "B," + tokens[1];
                // Tag 'B' for department
                context.write(new Text(deptId), new Text(deptName));
            }
        }
    }
```

JoinReducer.java

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
public class JoinReducer extends Reducer<Text, Text, Text, Text> {
@Override
protected void reduce(Text key, Iterable<Text> values, Context context) throws
IOException,
    InterruptedException {
        // key is the DeptID
        List<String> employeeList = new ArrayList<>();
        String deptName = "Unknown";
```

```
        // Separate employees and department name based on the tag
        for (Text val : values) {
            String[] parts = val.toString().split(",", 2);
            if (parts[0].equals("A")) { // It's an employee record
            employeeList.add(parts[1]); // "EmpID,EmpName"
            }
            else if (parts[0].equals("B")) { // It's a department record
            deptName = parts[1];
            }
        }
        // Perform the join if both sides are present
        if (!deptName.equals("Unknown") && !employeeList.isEmpty()) {
        for (String emp : employeeList) {
                String[] empData = emp.split(",");
                // Output: EmpID -> EmpName, DeptName
                context.write(new Text(empData[0]), new Text(empData[1] + "," +
                deptName));

            }
        }
    }
}
```

Output:

I/P

101, Alice, 501
102, Bob, 502
103, Charlie, 501
104, David, 503
105, Eve, 204

501, Sales
502, Technology
503, HR

O/P

101, Alice, Sales
102, Bob, Technology
103, Charlie, Sales
104, David HR

Exp No: 4B Date:	**Running a spark Application word count in Hadoop**

Aim

To compile and run a standalone Spark application that counts the occurrences of each word in a given text file on a Hadoop cluster.

Procedure

1: Save the Code

Save the provided Scala code into a file named WordCount.scala.

2: Set Up a Project for Building the JAR

You need a build tool to compile the Scala code and package it into a runnable JAR file. The most common tool for Scala is sbt (Scala Build Tool).

3: Compile and Package the Application

Use sbt to build your project. This command will compile the Scala code and create a JAR file.

4: Prepare Input on HDFS

Before you can run the application, you must upload your input data to a directory in the Hadoop Distributed File System (HDFS).
Create a directory on HDFS to store your input file: $ hdfs dfs -mkdir /user/wordcount_input

5: Submit the job to cluster

Use the spark-submit command to run your application. This command tells the cluster to execute the code in your JAR file.

Run the command from your terminal: $ spark-submit --class WordCount --master yarn --deploy-mode cluster /path/to/your/wordcount_jar_file.jar /user/wordcount_input /user/wordcount_output

6: View the Results

Once the job completes successfully, you can inspect the results on HDFS.

List the contents of the output directory: $ hdfs dfs -ls /user/wordcount_output The output will show each unique word followed by its count.

Program:

```
// Importing necessary libraries for Spark
import org.apache.spark.sql.SparkSession
import org.apache.spark.rdd.RDD
// Define a main object to serve as the entry point for the application
object WordCount {
// The main method where the program execution starts
def main(args: Array[String]): Unit = {
// Check if the correct number of command-line arguments are provided
if (args.length != 2) {
System.err.println("Usage: WordCount <input_path> <output_path>")
System.exit(1)
}
// Create a SparkSession, which is the entry point for all Spark functionality.
// The app name will be shown in the Spark web UI.

val spark = SparkSession.builder
.appName("Spark Word Count")
.getOrCreate()
// Get the input and output paths from the command-line arguments
val inputPath = args(0)
val outputPath = args(1)
// Read the text file from the specified input path on HDFS.
// This creates an RDD of strings, with each string representing a line from the file.
val lines: RDD[String] = spark.sparkContext.textFile(inputPath)

// Perform the word count logic using a series of transformations:
// 1. flatMap: Splits each line into individual words.
// 2. map: Converts each word into a key-value pair (word, 1).
// 3. reduceByKey: Sums the counts for each unique word.
val wordCounts = lines
.flatMap(line => line.split(" "))
.map(word => (word, 1))
.reduceByKey(_ + _)

// Save the final word counts to the specified output path on HDFS.
// The output will be a directory containing multiple part-files.
```

```
wordCounts.saveAsTextFile(outputPath)

// Stop the SparkSession to gracefully shut down the application
spark.stop()
}
}
```

Output:

if the input file "input.txt" contains the following text: "Hello world, hello Spark" The content of one of the output part files will look like this:

(hello,2)

(world,1)

(Spark,1)

The output will be saved to the HDFS directory specified as the second command-line argument to the spark-submit command, such as /user/wordcount_output

Exp No: 4C Date:	**Spark Manipulating RDD**

Aim

To understand and practice the manipulation of Resilient Distributed Datasets (RDDs) in Spark by performing fundamental operations such as creating RDDs from various sources and applying transformations and actions.

Procedure

1. **Initialize Spark**

Start a Spark shell session from your terminal. Use pyspark for Python or spark-shell for Scala, depending on which language you are using. The sc variable, which represents the SparkContext, will be available by default.

2. **Create RDDs**

You can create an RDD from an in-memory collection for small datasets or from an external file on HDFS for large datasets.

- Parallelized Collections: data = [1, 2, 3, 4, 5] and distData = sc.parallelize(data)
- External Datasets: distFile = sc.textFile("data.txt").

3. **Perform RDD Operations**

Apply transformations to create new RDDs and actions to trigger computation.

- Transformation: lines = sc.textFile("data.txt") and lineLengths = lines.map(lambda s: len(s))
- Action: totalLength = lineLengths.reduce(lambda a, b: a + b)

4. **Handle Functions and Classes**

When using Spark, be aware of how functions and classes are handled, especially with closures. You can use global functions, but for class methods, it is more efficient to capture required fields as local variables.

5. **Caution with Shared Variables**

Avoid modifying shared variables from within a transformation like foreach, as direct changes are not consistent across worker nodes.

Program

```
# Parallelized Collections data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)

# External Datasets
distFile = sc.textFile("data.txt")

# RDD Operations
lines = sc.textFile("data.txt")
lineLengths = lines.map( lambda s: len(s)) totalLength =
lineLengths.reduce( lambda a, b: a + b)
```

```
# Persist the RDD to cache for re-use l
ineLengths.persist()

# Example of a function
 """MyScript.py"""
if __name___== "__main__":
def myFunc(s):

words = s.split(" ")
return len(words)
sc = SparkContext(...)
sc.textFile("file.txt").map(myFunc)

// Example of a class (Scala)
class MyClass(field: String) {
def doStuff(rdd: RDD[String]): RDD[String] = {
// This captures 'field' as a local variable, which is good practice

val localField = field
rdd.map(s => localField + s)
}
}
```

Output:

Transformations are Lazy: Operations like parallelize, textFile, map, and persist are transformations. They do not produce any output directly, but instead define a plan for future computations.

• **Actions Trigger Output:** Only an action, such as reduce, will trigger the execution of the transformations and return a value to the driver program.

• **Expected Output:** The totalLength = lineLengths.reduce(lambda a, b: a + b) action would be the only part of the code to produce a result. It would sum the lengths of the lines in data.txt and assign the final integer value to the totalLength variable.

Exp No: 5A Date:	**Inverted Indexing in Spark (spark sequence algorithm problem)**

Aim

To implement inverted indexing in Spark using a sequence algorithm problem, demonstrating how Spark efficiently links words to the documents in which they occur.

Procedure

1. First, set up the Spark environment on the system by either installing Apache Spark or using PySpark through Python, ensuring all dependencies are properly configured to run Spark applications.
2. Prepare a small dataset of multiple text documents, each with a unique document identifier and content. This dataset will serve as the input to our Spark job and will be hard-coded inside the program for simplicity.
3. Load the dataset into Spark by parallelizing it as an RDD so that each line of text can be processed as an independent distributed element across different executors.
4. Split each line into two parts: the first token representing the document ID and the remaining tokens representing the words in the document. This step helps in distinguishing the source of each word.
5. Convert the text content into a sequence of (word, docID) pairs using flatMap, ensuring that every word occurrence is linked to its corresponding document identifier.
6. Apply groupByKey transformation to aggregate all document identifiers for each unique word, thereby constructing an inverted index that maps words to their occurrence list.
7. Format the aggregated results into a list structure that clearly displays all document IDs associated with each word, making the output human-readable.
8. Finally, collect the complete inverted index at the driver node, print it on the console, and verify that the output accurately reflects the mapping of words to the documents they belong to.

Program:

```
from pyspark import SparkContext
# Initialize Spark Context
sc = SparkContext("local", "SparkSequenceAlgorithm")
 # Input dataset: Document ID + Text
data = [
"doc1 Spark is fast and Spark is powerful for big data",
"doc2 Hadoop and Spark are important big data tools",
"doc3 Spark works well with Hadoop and Hive for analytics",
 "doc4 Machine learning with Spark is efficient and scalable",
 "doc5 Hive and Hadoop support Spark processing in clusters"
]
# Parallelize dataset into RDD
```

```
rdd = sc.parallelize(data)

# Convert lines into (word, docID) pairs

pairs = rdd.flatMap(lambda line: [(word.lower(), line.split()[0]) for word in line.split()[1:]])

# Group words to create inverted index
inverted_index =
pairs.groupByKey().mapValues(list)
# Collect and display results
output = inverted_index.collect()
for word, docs in sorted(output):
print (f "{word}: {docs}”)
# Stop Spark context
sc.stop()
```

Output

```
analytics: ['doc3']
and: ['doc1', 'doc2', 'doc4']
are: ['doc2']
big: ['doc1', 'doc2']
clusters: ['doc5']
data: ['doc1', 'doc2']
efficient: ['doc4']
fast: ['doc1']
for: ['doc1', 'doc3']
hadoop: ['doc2', 'doc3', 'doc5']
hive: ['doc3', 'doc5']
important: ['doc2']
is: ['doc1', 'doc4']

learning: ['doc4']
machine: ['doc4']
powerful: ['doc1']
processing: ['doc5']
scalable: ['doc4']
spark: ['doc1', 'doc1', 'doc2', 'doc3', 'doc4', 'doc5']
support: ['doc5']
tools: ['doc2']
well: ['doc3']
with: ['doc3', 'doc4']
works: ['doc3']
```

Result

The program has been executed successfully and the output was got.

Exp No: 5B Date:	**Inverted Indexing in spark (spark implementation of matrix algorithm)**

Aim

To implement a matrix algorithm using Apache Spark for efficient distributed computation and to observe the results of performing matrix operations at scale.

Procedure

1. First, configure the Spark environment by importing the required libraries and ensuring SparkContext and SparkSession are properly initialized.
2. Next, define the size and contents of two input matrices within the program so that they can be processed directly without external input files.
3. Represent the matrices as RDDs (Resilient Distributed Datasets) or DataFrames to leverage Spark's distributed data processing capabilities.
4. Partition the data adequately so that different blocks of the matrices can be handled in parallel across Spark worker nodes.
5. Perform the matrix multiplication logic by mapping over rows of the first matrix and columns of the second matrix, and then reduce the intermediate results to compute each cell value of the output matrix.
6. Collect the resulting matrix values from the distributed computation and format them back into a 2D matrix structure.
7. Print the final computed matrix in a structured format so that the correctness of the multiplication can be verified.
8. Finally, terminate the Spark context after confirming that the distributed execution of the matrix algorithm has successfully completed.

Program

```
from pyspark.sql import SparkSession

# Initialize Spark Session
spark = SparkSession.builder.appName("MatrixAlgorithm").getOrCreate() sc =
spark.sparkContext

# Define two matrices matrixA = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]

matrixB = [[9, 8, 7],
[6, 5, 4],
[3, 2, 1]]

# Convert to RDDs
rddA = sc.parallelize(enumerate(matrixA)) rddB = sc.parallelize(enumerate(matrixB))

# Transpose matrixB for easier multiplication
```

```
rddB_T = rddB.flatMap(lambda x: [(x[0], col_idx, val) for col_idx, val in
enumerate(x[1])])

# Perform multiplication result = rddA.flatMap(
lambda x: [((x[0], col_idx), sum([a*b for a, b in zip(x[1], [row[col_idx] for row in
matrixB])])) for col_idx in range(len(matrixB[0]))]
)
# Collect and format result
final_result = result.collect()
final_matrix = [[0 for _ in range(len(matrixB[0]))] for _ in range(len(matrixA))]

for ((i, j), value) in final_result: final_matrix[i][j] = value

print("Matrix A:") for row in matrixA:
print(row)

print("\nMatrix B:") for row in matrixB:
print(row)

print("\nResultant Matrix (A x B):") for row in final_matrix:
print(row) spark.stop()
```

OUTPUT:

```
Matrix A:
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

Matrix B:
[9, 8, 7]
[6, 5, 4]
[3, 2, 1]

Resultant Matrix (A x B):
[30, 24, 18]
[84, 69, 54]
[138, 114, 90]
```

Result

The program has been executed successfully and the output was got.

Exp No: 5C Date:	**Inverted Indexing in spark (spark sparlsql programming)**

Aim

To implement Spark SQL programming for performing structured queries on distributed datasets and to demonstrate how SQL operations can be executed efficiently in Apache Spark.

Procedure

1. First, initialize a SparkSession with SQL support enabled so that queries can be executed on structured data.
2. Next, create a sample dataset within the program itself by defining records in the form of tuples or dictionaries.
3. Convert the sample dataset into a Spark DataFrame, which provides a tabular representation of the data with named columns.
4. Register this DataFrame as a temporary SQL view so that it can be queried using Spark SQL statements.
5. Write multiple SQL queries such as selection, filtering, aggregation, and ordering to demonstrate the wide capabilities of Spark SQL.
6. Execute these queries and store the results into separate DataFrames for further inspection.
7. Display the results of the queries in tabular format to validate correctness and demonstrate Spark SQL's ability to handle structured queries.
8. Finally, close the Spark session after confirming that the SQL operations have executed successfully and outputs are verified.

Program

```
from pyspark.sql import SparkSession

# Initialize Spark Session
spark = SparkSession.builder.appName("SparkSQLExample").getOrCreate()

# Create sample dataset data = [
(1, "Alice", 23, "Computer Science"),
(2, "Bob", 29, "Mechanical"),
(3, "Cathy", 21, "Computer Science"),
(4, "David", 25, "Electrical"),
(5, "Eva", 22, "Mechanical"),
(6, "Frank", 28, "Computer Science"),
(7, "Grace", 24, "Electrical"),
(8, "Henry", 26, "Mechanical")
]

columns = ["ID", "Name", "Age", "Department"] df = spark.createDataFrame(data, columns)
df.createOrReplaceTempView("students")

query1 = spark.sql("SELECT * FROM students WHERE Age > 24")
query2 = spark.sql("SELECT Department, COUNT(*) as count FROM students GROUP BY
```

```
Department")
query3 = spark.sql("SELECT Name, Age FROM students ORDER BY Age ASC")
query4 = spark.sql("SELECT Department, AVG(Age) as avg_age FROM students GROUP
BY Department")

# Run Queries
print("Query 1: Students with Age > 24")
query1.show()

print("Query 2: Count of Students per Department")
query2.show()

print("Query 3: Students Ordered by Age")
query3.show()

print("Query 4: Average Age by Department")
query4.show()

spark.stop()
```

Output

```
Query 1: Students with Age > 24
+---+-----+---+-----------+
| ID| Name|Age| Department|
+---+-----+---+-----------+
|  2|  Bob| 29| Mechanical|
|  4|David| 25| Electrical|
|  6|Frank| 28|Computer Science|
|  8|Henry| 26| Mechanical|
+---+-----+---+-----------+


Query 2: Count of Students per Department
+-----------+-----+
| Department|count|
+-----------+-----+
| Mechanical|    3|
|Computer Science| 3|
|  Electrical|    2|
+-----------+-----+
```

```
Query 3: Students Ordered by Age
+-----+---+
| Name|Age|
+-----+---+
|Cathy| 21|
|  Eva| 22|
|Alice| 23|
|Grace| 24|
|David| 25|
|Henry| 26|
|Frank| 28|
|  Bob| 29|
+-----+---+


Query 4: Average Age by Department
+-----------+------------------+
| Department|           avg_age|
+-----------+------------------+
| Mechanical|26.0              |
|Computer Science|24.0         |
|  Electrical|24.5             |
+-----------+------------------+
```

Result

The program has been executed successfully and the output was got.

Exp No: 5D Date:	**Inverted Indexing in spark (Building spark streaming application)**

Aim

To build a Spark Streaming application that processes live data streams and demonstrates real- time word count computation using Apache Spark.

Procedure

1. First, set up a Spark session with streaming support so that continuous input data can be processed in real time.
2. Define a streaming source, such as a socket connection, to simulate live text data being received line by line.
3. Convert each incoming line of text into a stream of words by applying transformations like flatMap.
4. Apply the map operation to create key-value pairs where each word is associated with a count of one.
5. Use the reduceByKey operation on the streaming DataFrame or DStream to compute the running counts of each word.
6. Format the resulting stream so that the word counts are output in a readable tabular format after every batch interval.
7. Start the streaming context and keep it running long enough to capture and process multiple input batches.
8. Once the results are verified, stop the streaming gracefully, confirming that the Spark Streaming application has executed successfully.

Program

```
from pyspark.sql import SparkSession

from pyspark.streaming import StreamingContext

# Initialize Spark Session and Streaming Context

spark = SparkSession.builder.appName("SparkStreamingApp").getOrCreate() sc =
spark.sparkContext= interval

# Define input stream (socket text stream) lines = ssc.socketTextStream("localhost",
9999)

# Split lines into words

words = lines.flatMap(lambda line: line.split(" "))

# Map each word to (word, 1)

pairs = words.map(lambda word: (word, 1))

# Reduce by key to get word counts
```

```
wordCounts = pairs.reduceByKey(lambda x, y: x + y)

# Print results

wordCounts.pprint()

# Start Streaming ssc.start()

ssc.awaitTerminationOrTimeout(20)

# Run for 20 seconds

ssc.stop(stopSparkContext=True, stopGraceFully=True)
```

Input

```
hello world spark streaming
hello world
spark is powerful
streaming makes big data easy
```

Output

```
-------------------------------------------
Time: 2025-09-03 21:30:05
('hello', 2)
('world', 2)
('spark', 2)
('streaming', 2)

-------------------------------------------
Time: 2025-09-03 21:30:10
('is', 1)
('powerful', 1)
('makes', 1)
('big', 1)
('data', 1)
('easy', 1)

-------------------------------------------
Time: 2025-09-03 21:30:15
('hello', 2)
('world', 2)
('spark', 2)
('streaming', 2)
('is', 1)
('powerful', 1)
('makes', 1)
('big', 1)
('data', 1)
('easy', 1)
```

Result

The program has been executed successfully and the output was got.